Chapter 1 — Why Kubernetes Exists

From Servers to Containers to Orchestration

Chapter 1 — Why Kubernetes Exists

From Servers to Containers to Orchestration

1.1 The Real Problem Kubernetes Solves

Most beginners think Kubernetes is simply “a tool to run containers.” While technically true, this misses the real purpose of Kubernetes.

Kubernetes exists because modern systems became too large, too distributed, and too complex for humans to manage manually.

Kubernetes is really about:

• Automation

• Scale

• Resilience

• Distributed computing

• Self-healing systems

• Operating applications across many machines

The container is only the package. Kubernetes is the operating system that controls the entire data center.

One Kubernetes book describes Kubernetes as “the operating system of the cloud.”

1.2 Before Kubernetes — Traditional Servers

Before virtualization and containers became common, companies ran applications directly on physical servers.

Example:

• Server 1 → Web site

• Server 2 → Database

• Server 3 → Email

• Server 4 → Billing

Problems quickly appeared:

1. Wasted hardware

2. Poor isolation

3. Slow scaling

1.3 Virtual Machines Changed Everything

Virtualization introduced the idea of running multiple operating systems on one physical server.

Instead of:

1 Server = 1 Application

We now had:

1 Server = Many Virtual Machines

Benefits:

• Better hardware utilization

• Isolation

• Faster provisioning

1.4 Containers Enter the Scene

Containers changed the model again.

Instead of virtualizing hardware, containers virtualize the operating system. All containers share the host OS kernel.

Containers are:

• Lightweight

• Fast

• Portable

1.5 Docker Popularized Containers

Docker made containers practical and easy to use.

Example command:

docker run nginx

Docker introduced:

• Easy image creation

• Image registries

• Portable deployments

• Standardized workflows

1.6 The Explosion Problem

Docker solved packaging, but companies quickly faced a new problem.

Instead of running 5 large applications, they were now running:

• 500 containers

• 5,000 containers

• 50,000 containers

Humans could no longer manage them manually.

1.7 What Is Orchestration?

Orchestration means automated management of large numbers of containers.

Kubernetes automates:

• Deployments

• Scaling

• Self-healing

• Rolling updates

• Rollbacks

• Networking

• Storage

• Secrets

1.8 Kubernetes as a Distributed Operating System

Kubernetes transforms many machines into one logical computer.

Instead of managing 100 separate servers, you manage one Kubernetes cluster.

The cluster decides:

• Where workloads run

• How workloads recover

• How resources are allocated

1.9 Control Plane vs Worker Nodes

Control Plane:

• API Server

• Scheduler

• Controller Manager

• etcd

Worker Nodes:

• Run Pods

• Execute workloads

• Host containers

1.10 Pods — The Basic Unit of Kubernetes

Kubernetes does not deploy containers directly.

It deploys Pods.

A Pod is one or more tightly connected containers sharing:

• Networking

• Storage

• localhost communication

1.11 Desired State and Reconciliation

Traditional systems are imperative:

“Do this now.”

Kubernetes is declarative:

“This is how the system should look.”

Example:

replicas: 3

Kubernetes constantly checks reality and attempts to match the desired state.

This process is called reconciliation.

1.12 Self-Healing Infrastructure

If:

• A container crashes

• A node dies

• A process freezes

Kubernetes responds automatically:

• Restart container

• Replace Pod

• Move workload

• Remove unhealthy traffic targets

1.13 Autoscaling

Kubernetes supports automatic scaling.

Example:

CPU > 70%

Pods may scale:

3 Pods → 10 Pods

When traffic decreases:

10 Pods → 3 Pods

This is called Horizontal Pod Autoscaling (HPA).

1.14 Rolling Updates

Kubernetes supports zero-downtime rolling updates.

Process:

1. Start new Pod

2. Verify health

3. Shift traffic

4. Remove old Pod

1.15 Why Cloud Providers Love Kubernetes

All major cloud providers support Kubernetes:

• AWS → EKS

• GCP → GKE

• Azure → AKS

Kubernetes creates a standardized platform across clouds.

1.16 Kubernetes and Infrastructure Engineering

Kubernetes directly connects to:

• AWS

• GCP

• Terraform

• CI/CD

• Prometheus

• Service Meshes

• Observability

• Networking

• Security

• Platform Engineering

1.17 Final Mental Model

Linux:

Runs programs on one machine.

Kubernetes:

Runs programs across many machines.

Linux manages:

• Processes

• Memory

• Networking

• Storage

Kubernetes manages:

• Containers

• Pods

• Clusters

• Distributed networking

• Distributed storage

• Scaling

• Recovery

Chapter Summary

You now understand:

• Why Kubernetes was created

• Why containers became necessary

• Why orchestration exists

• The role of Pods

• Control plane vs worker nodes

• Declarative infrastructure

• Reconciliation

• Self-healing systems

• Rolling updates

• Autoscaling

Kubernetes is not just “a way to run containers.”

It is a distributed operating system for cloud infrastructure.